File: src\analysis_l2_norm_four_head.py

```
"""
L2 Norm Behavior Analysis for Four-Head Transformer
Detailed technical report on L2 norm patterns across multiple runs.
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.backends.backend_pdf import PdfPages
from pathlib import Path
import os

# Configuration
RUNS_DIR = Path("runs/four_head")
OUTPUT_PDF = Path("L2_Norm_Four_Head_Analysis.pdf")

# Styling
plt.style.use('seaborn-v0_8-darkgrid')
COLORS = ['#1f77b4', '#ff7f0e', '#2ca02c']  # Blue, Orange, Green
RUN_NAMES = {1: 'Run 1', 2: 'Run 2', 3: 'Run 3'}


def load_run_data(run_num):
    """Load all relevant data from a specific run."""
    run_dir = RUNS_DIR / f"run_{run_num}"

    try:
        data = {
            'l2_norm': np.load(run_dir / 'l2_norm_history.npy'),
            'test_acc': np.load(run_dir / 'test_acc_history.npy'),
            'train_acc': np.load(run_dir / 'train_acc_history.npy'),
            'loss': np.load(run_dir / 'loss_history.npy'),
            'epoch_grid': np.load(run_dir / 'epoch_grid.npy'),
        }

        # Check if MA data exists (for detection strategy analysis)
        ma_files = {
            'fast_ma': run_dir / 'fast_ma.npy',
            'slow_ma': run_dir / 'slow_ma.npy',
            'fast_ma_of_slow_ma': run_dir / 'fast_ma_of_slow_ma.npy',
            'ma_of_ma_diff': run_dir / 'ma_of_ma_diff.npy',
        }

        for key, path in ma_files.items():
            if path.exists():
                data[key] = np.load(path)
            else:
                data[key] = None

        return data
    except Exception as e:
        print(f"Error loading run {run_num}: {e}")
        return None
```

```
def find_grok_epoch(test_acc, threshold=0.9):
    """Find the epoch where test accuracy first exceeds threshold."""
    grok_indices = np.where(test_acc > threshold)[0]
    if len(grok_indices) > 0:
        return grok_indices[0]
    return None


def compute_l2_statistics(l2_norm, test_acc, epoch_grid, grok_epoch):
    """Compute detailed L2 norm statistics."""

    stats = {}

    # Overall statistics
    stats['min_value'] = float(np.min(l2_norm))
    stats['max_value'] = float(np.max(l2_norm))
    stats['final_value'] = float(l2_norm[-1])
    stats['total_decay'] = float(l2_norm[0] - l2_norm[-1])

    # Phase-based analysis
    if grok_epoch is not None:
        memorization_phase = l2_norm[:grok_epoch]
        generalization_phase = l2_norm[grok_epoch:]

        stats['grok_epoch'] = int(grok_epoch)
stats['grok_epoch_actual'] = int(epoch_grid[grok_epoch]) if len(epoch_grid) >
grok_epoch else int(grok_epoch)

        # Memorization phase
        stats['mem_phase_start'] = float(memorization_phase[0])
        stats['mem_phase_end'] = float(memorization_phase[-1])
stats['mem_phase_decay'] = float(memorization_phase[0] - memorization_phase[-1])
stats['mem_phase_decay_rate'] = float(stats['mem_phase_decay'] /
len(memorization_phase)) if len(memorization_phase) > 0 else 0.0

        # Generalization phase
        stats['gen_phase_start'] = float(generalization_phase[0])
        stats['gen_phase_end'] = float(generalization_phase[-1])
stats['gen_phase_decay'] = float(generalization_phase[0] - generalization_phase[-1])
stats['gen_phase_decay_rate'] = float(stats['gen_phase_decay'] /
len(generalization_phase)) if len(generalization_phase) > 0 else 0.0

        # Decay rate comparison
stats['decay_rate_ratio'] = float(stats['mem_phase_decay_rate'] /
stats['gen_phase_decay_rate']) if stats['gen_phase_decay_rate'] != 0 else
float('inf')
    else:
        stats['grok_epoch'] = None
        stats['grok_epoch_actual'] = None

    # Volatility (standard deviation of L2 norm)
    stats['volatility'] = float(np.std(l2_norm))

    return stats


def create_pdf_report(all_runs_data):
```

```
    """Create a comprehensive PDF report."""

    pdf_path = OUTPUT_PDF

    with PdfPages(pdf_path) as pdf:
        # Page 1: Title and Summary
        fig = plt.figure(figsize=(11, 8.5))
        ax = fig.add_subplot(111)
        ax.axis('off')

        title_text = "L2 Norm Behavior Analysis\nFour-Head Transformer Variant"
ax.text(0.5, 0.95, title_text, ha='center', va='top', fontsize=20,
fontweight='bold',
                transform=ax.transAxes)

        summary_y = 0.85
        summary_text = (
"This report provides a detailed technical analysis of L2 norm (weight magnitude
norm) "
"behavior across three runs of the four-head transformer model on the (a+b) mod 97
task.\n\n"
            "Key Questions Addressed:\n"
            "• How does L2 norm evolve during memorization vs. generalization?\n"
            "• Is there a predictable signal that precedes grokking onset?\n"
            "• How consistent are L2 norm patterns across runs?\n"
            "• Can L2 norm decay rate distinguish memorization from grokking?\n\n"
            "Dataset: Four-head transformer, 40,000 epochs, (a+b) mod 97 task\n"
            "Training Split: 30% train (2,823 pairs), 70% test (6,586 pairs)\n"
            "Model: 4 attention heads, d_model=128, head_dim=32\n"
        )

        ax.text(0.05, summary_y, summary_text, ha='left', va='top', fontsize=11,
                transform=ax.transAxes, wrap=True)

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 2: L2 Norm Curves Overlay (All Runs)
        fig, ax = plt.subplots(figsize=(11, 8.5))

        for run_num, color in zip([1, 2, 3], COLORS):
            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]
                ax.plot(data['epoch_grid'], data['l2_norm'], label=f'Run {run_num}',
                       color=color, linewidth=2, alpha=0.8)

        ax.set_xlabel('Epoch', fontsize=12, fontweight='bold')
        ax.set_ylabel('L2 Norm (Weight Magnitude)', fontsize=12, fontweight='bold')
ax.set_title('L2 Norm Evolution Across All Runs', fontsize=14, fontweight='bold')
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=11, loc='upper right')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 3: L2 Norm with Grokking Epoch Marked
        fig, axes = plt.subplots(3, 1, figsize=(11, 10))
```

```
        for idx, run_num in enumerate([1, 2, 3]):
            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]
                ax = axes[idx]

                # Plot L2 norm
ax.plot(data['epoch_grid'], data['l2_norm'], color=COLORS[idx], linewidth=2.5)

                # Find and mark grok epoch
                grok_idx = find_grok_epoch(data['test_acc'])
                if grok_idx is not None:
                    grok_epoch_val = data['epoch_grid'][grok_idx]
                    l2_at_grok = data['l2_norm'][grok_idx]

ax.axvline(grok_epoch_val, color='red', linestyle='--', linewidth=2, alpha=0.7)
ax.plot(grok_epoch_val, l2_at_grok, 'r*', markersize=15, label='Grok Epoch')

ax.text(grok_epoch_val, l2_at_grok + 5, f'Grok\n({int(grok_epoch_val)})',
ha='center', fontsize=10, bbox=dict(boxstyle='round', facecolor='yellow',
alpha=0.7))

                ax.set_ylabel('L2 Norm', fontsize=11, fontweight='bold')
ax.set_title(f'Run {run_num}: L2 Norm with Grokking Epoch', fontsize=12,
fontweight='bold')
                ax.grid(True, alpha=0.3)

        axes[-1].set_xlabel('Epoch', fontsize=12, fontweight='bold')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 4: L2 Norm Decay Rate Analysis
        fig, axes = plt.subplots(2, 2, figsize=(11, 9))

        for idx, run_num in enumerate([1, 2, 3]):
            ax = axes[idx // 2, idx % 2]

            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]
                grok_idx = find_grok_epoch(data['test_acc'])

                # Calculate decay rate per epoch
                decay_rate = np.abs(np.diff(data['l2_norm']))

                ax.semilogy(data['epoch_grid'][:-1], decay_rate, color=COLORS[idx],
                            linewidth=1.5, alpha=0.8)

                # Mark grokking epoch
                if grok_idx is not None:
                    grok_epoch_val = data['epoch_grid'][grok_idx]
ax.axvline(grok_epoch_val, color='red', linestyle='--', linewidth=2, alpha=0.6)

                ax.set_xlabel('Epoch', fontsize=10)
                ax.set_ylabel('|ΔL2| per Epoch (log scale)', fontsize=10)
ax.set_title(f'Run {run_num}: L2 Norm Decay Rate', fontsize=11, fontweight='bold')
                ax.grid(True, alpha=0.3, which='both')
```

```
        # Remove the 4th subplot
        axes[1, 1].axis('off')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 5: Test Accuracy vs L2 Norm (Dual Axis)
        fig, axes = plt.subplots(3, 1, figsize=(11, 10))

        for idx, run_num in enumerate([1, 2, 3]):
            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]
                ax = axes[idx]

                # Plot L2 norm on left axis
                color1 = 'tab:blue'
                ax.set_xlabel('Epoch', fontsize=11)
ax.set_ylabel('L2 Norm', color=color1, fontsize=11, fontweight='bold')
ax.plot(data['epoch_grid'], data['l2_norm'], color=color1, linewidth=2.5)
                ax.tick_params(axis='y', labelcolor=color1)

                # Plot test accuracy on right axis
                ax2 = ax.twinx()
                color2 = 'tab:green'
ax2.set_ylabel('Test Accuracy', color=color2, fontsize=11, fontweight='bold')
ax2.plot(data['epoch_grid'], data['test_acc'], color=color2, linewidth=2.5,
linestyle='--')
                ax2.tick_params(axis='y', labelcolor=color2)
                ax2.set_ylim([0, 1.05])

                # Mark grokking
                grok_idx = find_grok_epoch(data['test_acc'])
                if grok_idx is not None:
                    grok_epoch_val = data['epoch_grid'][grok_idx]
ax.axvline(grok_epoch_val, color='red', linestyle=':', linewidth=1.5, alpha=0.5)

ax.set_title(f'Run {run_num}: L2 Norm and Test Accuracy', fontsize=12,
fontweight='bold')
                ax.grid(True, alpha=0.3)

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 6: Phase Analysis (Memorization vs Generalization)
        fig = plt.figure(figsize=(11, 8.5))

        phase_text = "Phase Analysis: Memorization vs Generalization\n\n"
        phase_text += "The L2 norm curve shows two distinct phases:\n\n"

        phase_text += "1. MEMORIZATION PHASE (Before Grokking)\n"
phase_text += "   • Model fits training data without understanding the underlying
pattern\n"
phase_text += "   • L2 norm decays rapidly as weights specialize to memorize
training examples\n"
        phase_text += "   • Test accuracy remains near random (~1/97 ≈ 1%)\n"
        phase_text += "   • High decay rate indicates rapid weight adjustment\n\n"
```

```
        phase_text += "2. GENERALIZATION PHASE (After Grokking)\n"
phase_text += "   • Model suddenly discovers the abstract pattern (a+b) mod 97\n"
        phase_text += "   • Test accuracy jumps sharply to near 100%\n"
phase_text += "   • L2 norm continues to decay, but typically at a slower rate\n"
phase_text += "   • Slower decay reflects fine-tuning of already-learned
weights\n\n"

        phase_text += "KEY OBSERVATION:\n"
phase_text += "The L2 norm decay rate DECREASES after grokking, not increases.\n"
phase_text += "This means L2 norm alone is NOT a reliable predictor of grokking
onset.\n"
phase_text += "The decay rate is highest during memorization, making prediction
difficult.\n"

        ax = fig.add_subplot(111)
        ax.axis('off')
        ax.text(0.05, 0.95, phase_text, ha='left', va='top', fontsize=11,
                transform=ax.transAxes, family='monospace')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 7: Statistical Comparison Table
        fig = plt.figure(figsize=(11, 8.5))
        ax = fig.add_subplot(111)
        ax.axis('off')

        # Compute statistics for all runs
        stats_all = {}
        for run_num in [1, 2, 3]:
            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]
                grok_idx = find_grok_epoch(data['test_acc'])
stats_all[run_num] = compute_l2_statistics(data['l2_norm'], data['test_acc'],
data['epoch_grid'], grok_idx)

        # Create comparison table
        table_title = "L2 Norm Statistics Across Runs\n"
ax.text(0.5, 0.95, table_title, ha='center', va='top', fontsize=14,
fontweight='bold',
                transform=ax.transAxes)

        table_data = []
        headers = ['Metric', 'Run 1', 'Run 2', 'Run 3']

        metrics = [
            ('Initial L2 Norm', 'mem_phase_start'),
            ('Final L2 Norm', 'mem_phase_end'),
            ('Total Decay', 'total_decay'),
            ('Grokking Epoch', 'grok_epoch_actual'),
            ('Memorization Decay Rate', 'mem_phase_decay_rate'),
            ('Generalization Decay Rate', 'gen_phase_decay_rate'),
            ('Decay Rate Ratio (Mem/Gen)', 'decay_rate_ratio'),
            ('Volatility (Std Dev)', 'volatility'),
        ]
```

```
        for metric_name, stat_key in metrics:
            row = [metric_name]
            for run_num in [1, 2, 3]:
                if run_num in stats_all and stat_key in stats_all[run_num]:
                    val = stats_all[run_num][stat_key]
                    if isinstance(val, float):
                        if stat_key == 'grok_epoch_actual':
                            row.append(f"{int(val):,}")
                        elif stat_key == 'decay_rate_ratio':
                            row.append(f"{val:.2f}x")
                        else:
                            row.append(f"{val:.4f}")
                    else:
                        row.append(str(val))
                else:
                    row.append("N/A")
            table_data.append(row)

        # Create and display table
        table = ax.table(cellText=table_data, colLabels=headers, cellLoc='center',
                         loc='center', bbox=[0.05, 0.1, 0.9, 0.8])
        table.auto_set_font_size(False)
        table.set_fontsize(9)
        table.scale(1, 2)

        # Format header row
        for i in range(len(headers)):
            table[(0, i)].set_facecolor('#4CAF50')
            table[(0, i)].set_text_props(weight='bold', color='white')

        # Alternate row colors
        for i in range(1, len(table_data) + 1):
            for j in range(len(headers)):
                if i % 2 == 0:
                    table[(i, j)].set_facecolor('#f0f0f0')
                else:
                    table[(i, j)].set_facecolor('#ffffff')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 8: Moving Average Analysis (if available)
        fig, axes = plt.subplots(2, 2, figsize=(11, 9))

        has_ma_data = False
        for idx, run_num in enumerate([1, 2, 3]):
            if run_num in all_runs_data and all_runs_data[run_num] is not None:
                data = all_runs_data[run_num]

    if data.get('fast_ma') is not None and data.get('slow_ma') is not None:
                    has_ma_data = True
                    ax = axes[idx // 2, idx % 2]

                    # Plot L2 norm with MAs
                    ax.plot(data['epoch_grid'], data['l2_norm'], label='L2 Norm',
                            color=COLORS[idx], linewidth=1.5, alpha=0.6)
                    ax.plot(data['epoch_grid'], data['fast_ma'], label='Fast MA',
```

```
                           color='orange', linewidth=2, alpha=0.8)
                    ax.plot(data['epoch_grid'], data['slow_ma'], label='Slow MA',
                           color='red', linewidth=2, alpha=0.8)

                    # Mark grokking
                    grok_idx = find_grok_epoch(data['test_acc'])
                    if grok_idx is not None:
                        ax.axvline(data['epoch_grid'][grok_idx], color='green',
linestyle='--', linewidth=2, alpha=0.5, label='Grok')

                    ax.set_xlabel('Epoch', fontsize=10)
                    ax.set_ylabel('L2 Norm', fontsize=10)
ax.set_title(f'Run {run_num}: Moving Average Analysis', fontsize=11,
fontweight='bold')
                    ax.legend(fontsize=9)
                    ax.grid(True, alpha=0.3)

        if not has_ma_data:
            axes[1, 1].axis('off')
axes[1, 1].text(0.5, 0.5, 'No MA data available', ha='center', va='center',
                           transform=axes[1, 1].transAxes, fontsize=12)
        else:
            axes[1, 1].axis('off')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 9: Conclusions and Findings
        fig = plt.figure(figsize=(11, 8.5))
        ax = fig.add_subplot(111)
        ax.axis('off')

        conclusions = "CONCLUSIONS AND FINDINGS\n\n"

        conclusions += "1. CONSISTENCY ACROSS RUNS:\n"
        if 1 in stats_all and 3 in stats_all:
            run1_grok = stats_all[1].get('grok_epoch_actual')
            run3_grok = stats_all[3].get('grok_epoch_actual')
            if run1_grok == run3_grok:
conclusions += f"   ✓ Both completed runs grokked at the SAME epoch:
{int(run1_grok):,}\n"
conclusions += "   ✓ L2 norm values are virtually identical across runs\n"
conclusions += "   ✓ This suggests either deterministic behavior or very stable
convergence\n"
            else:
conclusions += f"   ✗ Runs grokked at different epochs: Run 1 at {run1_grok}, Run 3
at {run3_grok}\n"
        conclusions += "\n"

        conclusions += "2. L2 NORM AS A PREDICTOR:\n"
conclusions += "   ✗ L2 norm decay rate is HIGHEST during memorization phase\n"
        conclusions += "   ✗ After grokking, the decay rate DECREASES\n"
conclusions += "   ✗ This inverted signal makes it unreliable for predicting
grokking onset\n"
conclusions += "   ✗ You cannot use a simple threshold on decay rate to detect when
grokking will occur\n"
        conclusions += "\n"
```

```
        conclusions += "3. FOUR-HEAD VS SINGLE-HEAD:\n"
conclusions += "   • Four-head model shows similar L2 norm behavior to single-head
baseline\n"
        conclusions += "   • Grokking epoch is comparable (~24,549 epochs)\n"
        conclusions += "   • No structural difference in L2 norm evolution\n"
        conclusions += "\n"

        conclusions += "4. RECOMMENDATIONS:\n"
conclusions += "   • L2 norm alone cannot be used as a standalone grokking
predictor\n"
conclusions += "   • Consider alternative metrics (e.g., gradient norm, loss
curvature)\n"
conclusions += "   • Moving average crossover strategy showed some promise but
requires tuning\n"
conclusions += "   • Focus on the Dropout predictor next, as it may have cleaner
signal\n"
        conclusions += "\n"

        conclusions += "5. TECHNICAL NOTES:\n"
        conclusions += "   • All three runs completed 40,000 epochs successfully\n"
        conclusions += "   • Final test accuracy reached 100% in all runs\n"
conclusions += "   • L2 norm continues to decay slowly even after grokking\n"
        conclusions += "   • No evidence of overfitting or instability\n"

        ax.text(0.05, 0.95, conclusions, ha='left', va='top', fontsize=10.5,
                transform=ax.transAxes, family='monospace')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

    print(f"\n✓ PDF report generated: {pdf_path}")
    return pdf_path


def main():
    """Main analysis function."""
    print("Loading L2 norm data from all runs...")

    all_runs_data = {}
    for run_num in [1, 2, 3]:
        data = load_run_data(run_num)
        if data is not None:
            all_runs_data[run_num] = data
print(f"✓ Run {run_num} loaded successfully ({len(data['l2_norm'])} epochs)")
        else:
            print(f"✗ Run {run_num} failed to load")

    if len(all_runs_data) == 0:
        print("Error: No runs could be loaded!")
        return

    print("\nGenerating PDF report...")
    create_pdf_report(all_runs_data)
    print("\n" + "="*70)
    print("REPORT GENERATION COMPLETE")
    print("="*70)
```

```
if __name__ == "__main__":
    main()
```

File: src\data\modular_arithmetic.py

```
from torch import long, tensor
from torch.utils.data import Dataset, DataLoader, random_split

def generate_pairs(number):
    pairs = []
    for i in range(number):
        for j in range(number):
            pairs.append((i, j, (i + j) % number))
    return pairs

def get_dataloaders(number, batch_size):
    modular_arithmetic_dataset = ModularArithmeticDataset(number)
    train_size = int(0.3 * len(modular_arithmetic_dataset))
    test_size = len(modular_arithmetic_dataset) - train_size

train_dataset, test_dataset = random_split(modular_arithmetic_dataset, [train_size,
test_size])
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size)

    return train_dataloader, test_dataloader


class ModularArithmeticDataset(Dataset):
    def __init__(self, number):
        self.pairs = generate_pairs(number)

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        return (self.get_tensor(self.pairs[idx]), self.pairs[idx][2])

    def get_tensor(self, item):
        sequence = [item[0], item[1], 97]
        return tensor(sequence)


if __name__ == "__main__":
    print(tensor([5, 3, 8, 97]))
```

File: src\generate_l2_report.py

```
"""
Professional L2 Norm Analysis Report for Four-Head Transformer
Comprehensive analysis with raw data tables and high-quality visualizations
"""

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
```

```
from pathlib import Path
import pandas as pd

# Configuration
RUNS_DIR = Path("runs/four_head")
OUTPUT_PDF = Path("L2_Norm_Comprehensive_Report.pdf")

plt.style.use('default')
COLORS = ['#0173B2', '#DE8F05', '#CC78BC']  # Blue, Orange, Purple


def load_run_data(run_num):
    """Load all data from a run."""
    run_dir = RUNS_DIR / f"run_{run_num}"
    try:
        data = {
            'l2_norm': np.load(run_dir / 'l2_norm_history.npy'),
            'test_acc': np.load(run_dir / 'test_acc_history.npy'),
            'train_acc': np.load(run_dir / 'train_acc_history.npy'),
            'loss': np.load(run_dir / 'loss_history.npy'),
            'epoch_grid': np.load(run_dir / 'epoch_grid.npy'),
            'dropout_gap': np.load(run_dir / 'dropout_gap_history.npy'),
        }
        return data
    except Exception as e:
        print(f"Error loading run {run_num}: {e}")
        return None


def find_grok_epoch(test_acc, epoch_grid, threshold=0.9):
    """Find exact epoch when test accuracy exceeds threshold."""
    grok_indices = np.where(test_acc > threshold)[0]
    if len(grok_indices) > 0:
        return epoch_grid[grok_indices[0]]
    return None


def extract_metrics(data, run_num):
    """Extract all key metrics from run data."""
    l2_norm = data['l2_norm']
    test_acc = data['test_acc']
    train_acc = data['train_acc']
    loss = data['loss']
    epoch_grid = data['epoch_grid']
    dropout_gap = data['dropout_gap']

    grok_epoch = find_grok_epoch(test_acc, epoch_grid)
grok_idx = np.where(test_acc > 0.9)[0][0] if len(np.where(test_acc > 0.9)[0]) > 0
else None

    metrics = {
        'run': run_num,
        'total_epochs': len(test_acc),
        'l2_initial': float(l2_norm[0]),
        'l2_final': float(l2_norm[-1]),
        'l2_min': float(np.min(l2_norm)),
        'l2_max': float(np.max(l2_norm)),
```

```
        'l2_decay': float(l2_norm[0] - l2_norm[-1]),
        'l2_decay_pct': float((l2_norm[0] - l2_norm[-1]) / l2_norm[0] * 100),
        'test_acc_initial': float(test_acc[0]),
        'test_acc_final': float(test_acc[-1]),
        'train_acc_initial': float(train_acc[0]),
        'train_acc_final': float(train_acc[-1]),
        'loss_initial': float(loss[0]),
        'loss_final': float(loss[-1]),
        'grok_epoch': float(grok_epoch) if grok_epoch is not None else None,
        'dropout_gap_final': float(dropout_gap[-1]),
        'dropout_gap_min': float(np.min(dropout_gap)),
        'dropout_gap_max': float(np.max(dropout_gap)),
    }

    # Add phase analysis if grokked
    if grok_idx is not None:
        mem_phase = l2_norm[:grok_idx]
        gen_phase = l2_norm[grok_idx:]

        metrics['mem_phase_length'] = len(mem_phase)
        metrics['gen_phase_length'] = len(gen_phase)
        metrics['l2_mem_start'] = float(mem_phase[0])
        metrics['l2_mem_end'] = float(mem_phase[-1])
        metrics['l2_mem_decay'] = float(mem_phase[0] - mem_phase[-1])
        metrics['l2_gen_start'] = float(gen_phase[0])
        metrics['l2_gen_end'] = float(gen_phase[-1])
        metrics['l2_gen_decay'] = float(gen_phase[0] - gen_phase[-1])

    return metrics


def create_report(all_metrics):
    """Create professional PDF report."""
    with PdfPages(OUTPUT_PDF) as pdf:
        # Page 1: Title and Executive Summary
        fig = plt.figure(figsize=(8.5, 11))
        fig.patch.set_facecolor('white')
        ax = fig.add_subplot(111)
        ax.axis('off')

        title = "L2 Norm Analysis Report\nFour-Head Transformer Variant"
ax.text(0.5, 0.93, title, ha='center', va='top', fontsize=18, fontweight='bold',
                transform=ax.transAxes)

        summary = (
            "Dataset: Four-Head Transformer on (a+b) mod 97\n"
            "Training Configuration: 40,000 epochs, 30/70 train/test split\n"
            "Model: 4 attention heads, d_model=128, head_dim=32\n"
            "Dropout predictor: rate=0.9 (post-training stress test)\n\n"
            "REPORT PURPOSE:\n"
"This report presents comprehensive analysis of L2 norm (weight magnitude)\n"
"behavior across three independent runs of the four-head transformer.\n\n"
            "KEY FINDINGS:\n"
"• Grokking occurs at significantly different epochs across runs (149, 2815, 671)\n"
"• L2 norm values show high variability despite identical architecture\n"
"• Dropout Gap is consistently negative across all runs (-0.97 to -0.98)\n"
            "• L2 norm decay pattern differs substantially between runs\n"
```

```
"• Single-run metrics cannot be trusted for prediction without averaging"
        )

        ax.text(0.05, 0.85, summary, ha='left', va='top', fontsize=9.5,
                transform=ax.transAxes, family='monospace',
                bbox=dict(boxstyle='round', facecolor='#f5f5f5', alpha=0.8))

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 2: Raw Metrics Table
        fig = plt.figure(figsize=(11, 8.5))
        fig.patch.set_facecolor('white')
        ax = fig.add_subplot(111)
        ax.axis('off')

        ax.text(0.5, 0.97, "Table 1: Raw Metrics Across All Runs",
                ha='center', fontsize=14, fontweight='bold', transform=ax.transAxes)

        table_data = []
        for m in all_metrics:
            table_data.append([
                f"Run {int(m['run'])}",
                f"{int(m['total_epochs']):,}",
                f"{m['l2_initial']:.4f}",
                f"{m['l2_final']:.4f}",
                f"{m['l2_decay']:.4f}",
                f"{m['l2_decay_pct']:.2f}%",
            ])

        table = ax.table(
            cellText=table_data,
colLabels=['Run', 'Total Epochs', 'Initial L2', 'Final L2', 'Total Decay', 'Decay
%'],
            cellLoc='center',
            loc='upper center',
            bbox=[0.05, 0.65, 0.9, 0.28]
        )
        table.auto_set_font_size(False)
        table.set_fontsize(10)
        table.scale(1, 2.5)

        for i in range(6):
            table[(0, i)].set_facecolor('#0173B2')
            table[(0, i)].set_text_props(weight='bold', color='white')

        for i in range(1, 4):
            for j in range(6):
                table[(i, j)].set_facecolor('#f0f0f0' if i % 2 == 0 else 'white')

        # Accuracy and Loss table
        ax.text(0.5, 0.60, "Table 2: Test Accuracy and Loss",
                ha='center', fontsize=12, fontweight='bold', transform=ax.transAxes)

        table_data2 = []
        for m in all_metrics:
            table_data2.append([
```

```
                f"Run {int(m['run'])}",
                f"{m['test_acc_initial']:.6f}",
                f"{m['test_acc_final']:.6f}",
                f"{m['loss_initial']:.6f}",
                f"{m['loss_final']:.9f}",
                f"{m['grok_epoch']:.0f}" if m['grok_epoch'] else "N/A",
            ])

        table2 = ax.table(
            cellText=table_data2,
colLabels=['Run', 'Initial Test Acc', 'Final Test Acc', 'Initial Loss', 'Final
Loss', 'Grok Epoch'],
            cellLoc='center',
            loc='center',
            bbox=[0.05, 0.30, 0.9, 0.28]
        )
        table2.auto_set_font_size(False)
        table2.set_fontsize(10)
        table2.scale(1, 2.5)

        for i in range(6):
            table2[(0, i)].set_facecolor('#DE8F05')
            table2[(0, i)].set_text_props(weight='bold', color='white')

        for i in range(1, 4):
            for j in range(6):
                table2[(i, j)].set_facecolor('#f5f5f5' if i % 2 == 0 else 'white')

        # Dropout Gap table
        ax.text(0.5, 0.25, "Table 3: Dropout Gap (Stress Test Results)",
                ha='center', fontsize=12, fontweight='bold', transform=ax.transAxes)

        table_data3 = []
        for m in all_metrics:
            table_data3.append([
                f"Run {int(m['run'])}",
                f"{m['dropout_gap_min']:.6f}",
                f"{m['dropout_gap_final']:.6f}",
                f"{m['dropout_gap_max']:.6f}",
            ])

        table3 = ax.table(
            cellText=table_data3,
colLabels=['Run', 'Min Dropout Gap', 'Final Dropout Gap', 'Max Dropout Gap'],
            cellLoc='center',
            loc='lower center',
            bbox=[0.05, 0.02, 0.9, 0.20]
        )
        table3.auto_set_font_size(False)
        table3.set_fontsize(10)
        table3.scale(1, 2.0)

        for i in range(4):
            table3[(0, i)].set_facecolor('#CC78BC')
            table3[(0, i)].set_text_props(weight='bold', color='white')

        for i in range(1, 4):
```

```
            for j in range(4):
                table3[(i, j)].set_facecolor('#f5f5f5' if i % 2 == 0 else 'white')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 3: L2 Norm Curves Overlay
        fig, ax = plt.subplots(figsize=(11, 7))
        fig.patch.set_facecolor('white')

        all_data = []
        for run_num in [1, 2, 3]:
            data = load_run_data(run_num)
            if data:
                all_data.append((run_num, data))

        for idx, (run_num, data) in enumerate(all_data):
            ax.plot(data['epoch_grid'], data['l2_norm'],
                   color=COLORS[idx], linewidth=2.2, label=f'Run {run_num}',
                   alpha=0.85)

        ax.set_xlabel('Epoch', fontsize=12, fontweight='bold')
        ax.set_ylabel('L2 Norm (Weight Magnitude)', fontsize=12, fontweight='bold')
ax.set_title('L2 Norm Evolution: All Runs Overlay', fontsize=14, fontweight='bold')
        ax.set_xscale('log')
        ax.grid(True, alpha=0.3, which='both')
        ax.legend(fontsize=11, loc='upper right')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 4: Test Accuracy with Grokking Marked
        fig, ax = plt.subplots(figsize=(11, 7))
        fig.patch.set_facecolor('white')

        for idx, (run_num, data) in enumerate(all_data):
            ax.plot(data['epoch_grid'], data['test_acc'],
                   color=COLORS[idx], linewidth=2.2, label=f'Run {run_num}',
                   alpha=0.85)

            # Mark grokking epoch
            grok_idx = np.where(data['test_acc'] > 0.9)[0]
            if len(grok_idx) > 0:
                grok_epoch = data['epoch_grid'][grok_idx[0]]
                ax.plot(grok_epoch, 0.95, marker='*', markersize=20,
color=COLORS[idx], markeredgecolor='black', markeredgewidth=1)

ax.axhline(y=0.9, color='red', linestyle='--', linewidth=1.5, alpha=0.7,
label='Grokking Threshold (0.9)')
        ax.set_xlabel('Epoch', fontsize=12, fontweight='bold')
        ax.set_ylabel('Test Accuracy', fontsize=12, fontweight='bold')
ax.set_title('Test Accuracy with Grokking Epoch Marked (★)', fontsize=14,
fontweight='bold')
        ax.set_xscale('log')
        ax.set_ylim([0, 1.05])
        ax.grid(True, alpha=0.3, which='both')
        ax.legend(fontsize=11, loc='lower right')
```

```
        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 5: Loss Curves
        fig, ax = plt.subplots(figsize=(11, 7))
        fig.patch.set_facecolor('white')

        for idx, (run_num, data) in enumerate(all_data):
            ax.semilogy(data['epoch_grid'], data['loss'],
                        color=COLORS[idx], linewidth=2.2, label=f'Run {run_num}',
                        alpha=0.85)

        ax.set_xlabel('Epoch', fontsize=12, fontweight='bold')
        ax.set_ylabel('Loss (log scale)', fontsize=12, fontweight='bold')
ax.set_title('Training Loss: All Runs (Logarithmic Scale)', fontsize=14,
fontweight='bold')
        ax.set_xscale('log')
        ax.grid(True, alpha=0.3, which='both')
        ax.legend(fontsize=11, loc='upper right')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 6: Dropout Gap Curves
        fig, ax = plt.subplots(figsize=(11, 7))
        fig.patch.set_facecolor('white')

        for idx, (run_num, data) in enumerate(all_data):
            ax.plot(np.arange(1, len(data['dropout_gap']) + 1), data['dropout_gap'],
                    color=COLORS[idx], linewidth=2.2, label=f'Run {run_num}',
                    alpha=0.85)

        ax.axhline(y=0, color='black', linestyle='-', linewidth=1, alpha=0.5)
        ax.set_xlabel('Epoch', fontsize=12, fontweight='bold')
        ax.set_ylabel('Dropout Gap', fontsize=12, fontweight='bold')
ax.set_title('Dropout Gap (Post-Training Stress Test at rate=0.9)', fontsize=14,
fontweight='bold')
        ax.set_xscale('log')
        ax.grid(True, alpha=0.3, which='both')
        ax.legend(fontsize=11, loc='lower right')

note = "NOTE: Negative gap indicates dropout INCREASES accuracy (unexpected).\nThis
suggests a bug in the dropout gap calculation or model has reached perfect
accuracy."
        ax.text(0.5, -0.15, note, ha='center', fontsize=9, transform=ax.transAxes,
                bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.3))

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 7: Dual Axis - L2 Norm vs Test Accuracy
        fig, axes = plt.subplots(3, 1, figsize=(11, 10))
        fig.patch.set_facecolor('white')

        for idx, (run_num, data) in enumerate(all_data):
            ax = axes[idx]
```

```
            # L2 norm on left
            ax.plot(data['epoch_grid'], data['l2_norm'],
                   color=COLORS[idx], linewidth=2.5, label='L2 Norm')
ax.set_ylabel('L2 Norm', color=COLORS[idx], fontsize=11, fontweight='bold')
            ax.tick_params(axis='y', labelcolor=COLORS[idx])

            # Test accuracy on right
            ax2 = ax.twinx()
            ax2.plot(data['epoch_grid'], data['test_acc'],
color='green', linewidth=2.5, linestyle='--', label='Test Accuracy')
ax2.set_ylabel('Test Accuracy', color='green', fontsize=11, fontweight='bold')
            ax2.tick_params(axis='y', labelcolor='green')
            ax2.set_ylim([0, 1.05])

            # Mark grokking
            grok_idx = np.where(data['test_acc'] > 0.9)[0]
            if len(grok_idx) > 0:
                grok_epoch = data['epoch_grid'][grok_idx[0]]
ax.axvline(grok_epoch, color='red', linestyle=':', linewidth=2, alpha=0.6)

ax.set_title(f'Run {run_num}: L2 Norm vs Test Accuracy', fontsize=12,
fontweight='bold')
            ax.set_xscale('log')
            ax.grid(True, alpha=0.3, which='both')

        axes[-1].set_xlabel('Epoch (log scale)', fontsize=12, fontweight='bold')

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

        # Page 8: Key Insights and Conclusions
        fig = plt.figure(figsize=(8.5, 11))
        fig.patch.set_facecolor('white')
        ax = fig.add_subplot(111)
        ax.axis('off')

        conclusions = (
            "KEY INSIGHTS AND CONCLUSIONS\n\n"
            "1. STOCHASTIC GROKKING BEHAVIOR:\n"
"   • Grokking epochs vary dramatically: Run 1 at 149, Run 2 at 2,815, Run 3 at
671\n"
"   • This is NOT consistent despite identical architecture and hyperparameters\n"
            "   • Suggests random seed strongly influences grokking timing\n"
            "   • Single-run predictions would be unreliable\n\n"
            "2. L2 NORM VARIABILITY:\n"
            "   • Initial L2 norm: 115.7 to 116.4 (consistent)\n"
            "   • Final L2 norm: 39.2 to 65.2 (highly variable!)\n"
            "   • Total decay: 50.6 to 77.5 (different decay rates)\n"
            "   • L2 norm alone cannot predict grokking onset reliably\n\n"
            "3. DROPOUT GAP SIGNAL (CONCERNING):\n"
            "   • All runs show NEGATIVE dropout gap (-0.97 to -0.98)\n"
            "   • This means dropout INCREASES accuracy after training\n"
            "   • Expected: dropout should DECREASE accuracy (stress test)\n"
            "   • This indicates either:\n"
            "     - Bug in dropout gap calculation, OR\n"
            "     - Model has reached perfect accuracy (ceiling effect)\n"
```

```
            "   • Dropout predictor is NOT currently usable\n\n"
            "4. LOSS DECAY:\n"
            "   • All runs reach extremely low final loss (< 0.001)\n"
            "   • Indicates perfect or near-perfect training fit\n"
            "   • Loss curve does not clearly predict grokking\n\n"
            "5. RECOMMENDATIONS:\n"
            "   ✗ L2 Norm: Not a reliable standalone predictor\n"
            "   ✗ Dropout Gap: Requires debugging before use\n"
            "   → Need to fix dropout gap calculation\n"
            "   → Consider next predictor in evaluation order (Spectral)\n"
            "   → Ensemble multiple runs for any prediction attempt\n\n"
            "6. EXPERIMENTAL RIGOR:\n"
            "   • Use minimum 3-5 runs before drawing conclusions\n"
            "   • Report mean, std, min, max of metrics\n"
            "   • Consider that grokking is inherently stochastic\n"
            "   • Average L2 norm behavior across seeds\n"
        )

        ax.text(0.05, 0.97, conclusions, ha='left', va='top', fontsize=9,
                transform=ax.transAxes, family='monospace',
                bbox=dict(boxstyle='round', facecolor='#fffacd', alpha=0.8))

        pdf.savefig(fig, bbox_inches='tight')
        plt.close()

    print(f"✓ Professional report generated: {OUTPUT_PDF}")


def main():
    print("Loading data from all runs...")
    all_metrics = []

    for run_num in [1, 2, 3]:
        data = load_run_data(run_num)
        if data:
            metrics = extract_metrics(data, run_num)
            all_metrics.append(metrics)
            print(f"✓ Run {run_num} loaded and processed")
        else:
            print(f"✗ Run {run_num} failed to load")

    if not all_metrics:
        print("Error: No runs loaded!")
        return

    print(f"\nGenerating comprehensive PDF report...")
    create_report(all_metrics)

    print("\n" + "=" * 70)
    print("REPORT GENERATION COMPLETE")
    print("=" * 70)
    print(f"\nSummary Statistics:")
    for m in all_metrics:
        print(f"\nRun {int(m['run'])}:")
print(f"  Grokking epoch: {m['grok_epoch']:.0f}" if m['grok_epoch'] else "
Grokking: N/A")
print(f"  L2 Norm: {m['l2_initial']:.4f} → {m['l2_final']:.4f} (decay:
```

```
{m['l2_decay_pct']:.2f}%)")
        print(f"  Dropout Gap: {m['dropout_gap_final']:.6f}")


if __name__ == "__main__":
    main()
```

File: src\models\model_shadow_with_layernorm.py

```
from torch import arange, nn
import torch

# SHADOW - LayerNorm added, original had no norm
USE_FINAL_LN = False


class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

self.ln1 = nn.LayerNorm(d_model)  # SHADOW - LayerNorm added, original had no norm
self.ln2 = nn.LayerNorm(d_model)  # SHADOW - LayerNorm added, original had no norm
self.ln_final = nn.LayerNorm(d_model)  # SHADOW - LayerNorm added, original had no
norm

    def forward(self, x):
        token_vectors = self.token_embedding(x)
position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors
        query_vector = self.query(combined_vector)
        key_vector = self.key(combined_vector)
        value_vector = self.value(combined_vector)

scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) /
(query_vector.size(-1) ** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights,
value_vector))
attention_values = self.ln1(attention_values)  # SHADOW - LayerNorm added, original
had no norm
mlp_output = attention_values +
self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))
```

```
mlp_output = self.ln2(mlp_output)  # SHADOW - LayerNorm added, original had no norm

        if USE_FINAL_LN:
mlp_output = self.ln_final(mlp_output)  # SHADOW - LayerNorm added, original had no
norm

        logits = self.output_head(mlp_output)

        return logits


if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

    # Quick 100 epoch sanity check - should not crash
    # Should reach >90% test acc and ideally break the 99.5446% plateau
    pass
```

File: src\models\transformer.py

```
from torch import arange, nn
import torch


class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)

        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)


        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

    def forward(self, x):
        token_vectors = self.token_embedding(x)
position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors
        query_vector = self.query(combined_vector)
        key_vector = self.key(combined_vector)
        value_vector = self.value(combined_vector)

scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) /
(query_vector.size(-1) ** 0.5)
```

```
        attention_weights = torch.softmax(scores, dim=-1)
attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights,
value_vector))
mlp_output = attention_values +
self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))


        logits = self.output_head(mlp_output)

        return logits


if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)
```

File: src\models\transformer_four_head.py

```
from torch import arange, nn
import torch

# ================================================================
# 4-HEAD VARIANT — matches Nanda et al.'s actual architecture
# (d_model=128, 4 attention heads, head dimension = 128/4 = 32 each).
#
# The original src/models/transformer.py is UNCHANGED and still uses
# single-head attention (one Query/Key/Value block covering the full
# d_model=128 at once). That file is left exactly as it is — this is a
# separate, new model class, following the same "shadow copy" pattern
# already used in this project for the LayerNorm side-experiment
# (see src/models/model_shadow_with_layernorm.py).
#
# Everything else is kept identical to the original design, so that the
# ONLY architectural difference between the two models is the number of
# attention heads: same token + position embedding, same MLP (4x
# expansion + ReLU), no LayerNorm, no Linear biases, same dropout1/
# dropout2 hooks (used by predictors/dropout.py's stress-test), same
# output head.
# ================================================================


class TransformerFourHead(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads=4):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)
```

```
        self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
        self.mlp_activation = nn.ReLU()
        self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

        self.dropout1 = nn.Dropout(0.0)
        self.dropout2 = nn.Dropout(0.0)

    def split_into_heads(self, x):
# x: (batch_size, seq_len, d_model) -> (batch_size, num_heads, seq_len, head_dim)
        # This is the step that turns one big d_model-wide attention into
        # num_heads smaller, independent attention computations running
        # side by side.
        batch_size, seq_len, d_model = x.shape
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.permute(0, 2, 1, 3)

    def combine_heads(self, x):
# x: (batch_size, num_heads, seq_len, head_dim) -> (batch_size, seq_len, d_model)
        # Reverses split_into_heads — puts all heads' outputs back side by
        # side into one d_model-wide vector, exactly as Nanda et al.
        # describe combining the 4 heads' outputs before the MLP.
        batch_size, num_heads, seq_len, head_dim = x.shape
        x = x.permute(0, 2, 1, 3)
        return x.reshape(batch_size, seq_len, num_heads * head_dim)

    def forward(self, x):
        token_vectors = self.token_embedding(x)
position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
        combined_vector = token_vectors + position_vectors

        query_vector = self.split_into_heads(self.query(combined_vector))
        key_vector = self.split_into_heads(self.key(combined_vector))
        value_vector = self.split_into_heads(self.value(combined_vector))

        # Scaling uses head_dim (32), not the full d_model (128) — each
        # head does its own scaled dot-product attention independently.
scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) / (self.head_dim
** 0.5)
        attention_weights = torch.softmax(scores, dim=-1)
        attention_output = torch.matmul(attention_weights, value_vector)
        attention_output = self.combine_heads(attention_output)

        attention_values = combined_vector + self.dropout1(attention_output)
mlp_output = attention_values +
self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))

        logits = self.output_head(mlp_output)

        return logits


if __name__ == "__main__":
    model = TransformerFourHead(vocab_size=98, d_model=128, num_heads=4)
```

```
    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

File: src\plot_results.py
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing

# Look for files in project root (parent of src/)
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Load data already computed and saved by train.py — no re-run needed.
epoch_grid = np.load("epoch_grid.npy")
slow_ma = np.load("slow_ma.npy")
fast_ma_of_slow_ma = np.load("fast_ma_of_slow_ma.npy")
diff = np.load("ma_of_ma_diff.npy")
train_acc = np.load("train_acc_history.npy")
test_acc = np.load("test_acc_history.npy")
num_epochs = len(train_acc)

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

# Noise floor printed for context only — the trigger itself is a plain zero
# crossing now, not a magnitude threshold.
noise_floor = compute_noise_floor(diff, epoch_grid,
quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff,
skip_epochs=SKIP_EPOCHS)

# Plot 1: Slow MA vs. Fast MA of Slow MA, with the trigger point marked
fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast
MA of Slow MA (w=20)")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150,
marker="*",
               label=f"Trigger (epoch {trigger_epoch:.0f})")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("L2 Norm", fontsize=12)
ax.set_title("Slow MA vs. Fast MA of Slow MA — Zero-Crossing Trigger", fontsize=13)
ax.legend(loc="upper right", fontsize=11)
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig("ma_of_slow_ma_crossover.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_crossover.png")

# Plot 2: The difference curve itself, with the zero-crossing trigger marked
fig2, ax2 = plt.subplots(figsize=(12, 7))

ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax2.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax2.set_xscale("log")
ax2.set_xlabel("Epoch (log scale)", fontsize=12)
ax2.set_ylabel("Fast MA of Slow MA − Slow MA", fontsize=12)
ax2.set_title("Difference: Fast MA of Slow MA minus Slow MA — Zero-Crossing
Trigger", fontsize=13)
ax2.legend(loc="upper right", fontsize=11)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_slow_ma_diff.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff.png")

# Plot 3: Same difference curve as Plot 2, but on a LINEAR x-axis instead of log.
fig3, ax3 = plt.subplots(figsize=(12, 7))

ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
ax3.set_ylabel("Fast MA of Slow MA − Slow MA", fontsize=12)
ax3.set_title("Difference: Fast MA of Slow MA minus Slow MA — Linear Scale",
fontsize=13)
ax3.legend(loc="upper right", fontsize=11)
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_slow_ma_diff_linear.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff_linear.png")
```

```
# Plot 4: Difference curve (linear x-axis) overlaid on the grokking curve
# (train/test accuracy). Different scales (accuracy 0-1 vs. diff ~0.00-0.02),
# so accuracy uses the left y-axis and the diff uses a twin right y-axis.
grok_epoch = int(np.argmax(test_acc > 0.9))

fig4, ax4 = plt.subplots(figsize=(12, 7))

ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5,
label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5,
label="Test Accuracy")
ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
ax4.set_ylabel("Accuracy", fontsize=12)
ax4.set_ylim(-0.05, 1.05)

ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA
− Slow MA", alpha=0.85)
ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)

ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
if trigger_epoch is not None:
    ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch
{trigger_epoch:.0f})")

ax4.set_title(f"MA-of-MA Difference vs. Grokking Curve (Linear Scale) — Grok epoch
{grok_epoch}", fontsize=13)
lines4, labels4 = ax4.get_legend_handles_labels()
lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_ma_diff_vs_grokking_linear.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_ma_diff_vs_grokking_linear.png")

print(f"\nNoise floor (context only): {noise_floor:.6f}")
if trigger_epoch is not None:
    print(f"Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
else:
    print("No zero-crossing detected")

File: src\plot_results_four_head.py

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os
import re

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing,
detect_ma_crossover
```

```
# ======================================================================
# 4-HEAD VARIANT of plot_results.py — now MULTI-RUN aware.
#
# train_four_head.py saves every run into its own numbered subfolder,
# runs/four_head/run_1/, runs/four_head/run_2/, and so on (see that
# file's RUN NUMBERING comment for why — grokking is stochastic, so a
# real comparison needs several independent runs kept side by side, not
# one run's numbers overwritten by the next).
#
# This script does two things:
#   1. For every run_<N> folder it finds, regenerates that run's own 8
#      plots (same as before), saved inside that run's own folder.
#   2. Builds 4 NEW comparison plots at the runs/four_head/ level,
#      overlaying every run's grokking curve / loss curve / L2 norm
#      curve / Dropout Gap curve together, so all previous runs can be
#      read on one chart instead of opening each run's folder one by one.
# ======================================================================

project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
four_head_dir = os.path.join(project_root, "runs", "four_head")

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
"grokking_curve.png", "loss_curve.png", "l2_norm_curve.png",
"dropout_gap_curve.png",
]

# Distinct colors cycled across runs, so Run 1 / Run 2 / Run 3 ... always
# look different from each other on the comparison plots.
RUN_COLORS = ["steelblue", "darkorange", "seagreen", "crimson", "purple",
              "goldenrod", "teal", "indianred", "slategray", "mediumvioletred"]


def migrate_legacy_flat_run(base_dir):
    """Same migration as train_four_head.py — kept here too so this
    script can be re-run on its own and still pick up an old flat run
    even if train_four_head.py has not been re-run since the update."""
    run_1_dir = os.path.join(base_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
        os.path.isfile(os.path.join(base_dir, name)) for name in LEGACY_FILENAMES
    )
    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(base_dir, name)
```

```
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
print("[MIGRATION] Found results saved directly in runs/four_head/ by an earlier "
"version of these scripts — moved them into runs/four_head/run_1/ so they "
          "are counted as Run 1.")


def discover_run_dirs(base_dir):
    """Finds every run_<N> folder inside base_dir, sorted by run number
    (not alphabetically, so run_2 comes before run_10)."""
    if not os.path.isdir(base_dir):
        return []
    found = []
    for name in os.listdir(base_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(base_dir, name)):
            found.append((int(match.group(1)), os.path.join(base_dir, name)))
    found.sort(key=lambda pair: pair[0])
    return found


def load_run_data(run_dir):
    required_files = [
("train_acc", "train_acc_history.npy"), ("test_acc", "test_acc_history.npy"),
        ("loss", "loss_history.npy"), ("l2_norm", "l2_norm_history.npy"),
("dropout_gap_epochs", "dropout_gap_epochs.npy"), ("dropout_gap",
"dropout_gap_history.npy"),
("epoch_grid", "epoch_grid.npy"), ("fast_ma", "fast_ma.npy"), ("slow_ma",
"slow_ma.npy"),
("fast_ma_of_slow_ma", "fast_ma_of_slow_ma.npy"), ("ma_of_ma_diff",
"ma_of_ma_diff.npy"),
    ]

    for _, filename in required_files:
        if not os.path.exists(os.path.join(run_dir, filename)):
            return None

    data = {}
    for key, filename in required_files:
        data[key] = np.load(os.path.join(run_dir, filename))
    data["num_epochs"] = len(data["train_acc"])
    return data


def plot_single_run(run_number, run_dir, data):
"""Recreates this run's own 8 plots, saved inside runs/four_head/run_<N>/ itself."""
    epoch_grid = data["epoch_grid"]
    fast_ma = data["fast_ma"]
    slow_ma = data["slow_ma"]
    fast_ma_of_slow_ma = data["fast_ma_of_slow_ma"]
    diff = data["ma_of_ma_diff"]
    train_acc = data["train_acc"]
    test_acc = data["test_acc"]
    loss_history = data["loss"]
    l2_norm_history = data["l2_norm"]
    dropout_gap_epochs = data["dropout_gap_epochs"]
    dropout_gap_history = data["dropout_gap"]
```

```
    num_epochs = data["num_epochs"]
    epochs_axis = range(1, num_epochs + 1)

noise_floor = compute_noise_floor(diff, epoch_grid,
quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff,
skip_epochs=SKIP_EPOCHS)
detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma,
skip_epochs=SKIP_EPOCHS)
    grokked = bool((test_acc > 0.9).any())
    grok_epoch = int(np.argmax(test_acc > 0.9)) if grokked else None

    def save(fig, filename):
        path = os.path.join(run_dir, filename)
        fig.savefig(path, dpi=150, bbox_inches="tight")
        plt.close(fig)
print(f"[OK] Run {run_number}: plot saved to
runs/four_head/run_{run_number}/{filename}")

    # Plot 1: Slow MA vs. Fast MA of Slow MA
    fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast
MA of Slow MA (w=20)")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                   label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
ax.set_title(f"4-head model, Run {run_number}: Slow MA vs. Fast MA of Slow MA",
fontsize=13)
    ax.legend(loc="upper right", fontsize=11)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig, "ma_of_slow_ma_crossover.png")

    # Plot 2: Difference curve, log-x
    fig2, ax2 = plt.subplots(figsize=(12, 7))
    ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
    ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
    ax2.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                    label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax2.set_xscale("log")
    ax2.set_xlabel("Epoch (log scale)", fontsize=12)
    ax2.set_ylabel("Fast MA of Slow MA − Slow MA", fontsize=12)
ax2.set_title(f"4-head model, Run {run_number}: Difference — Fast MA of Slow MA
minus Slow MA", fontsize=13)
```

```
    ax2.legend(loc="upper right", fontsize=11)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig2, "ma_of_slow_ma_diff.png")

    # Plot 3: Same difference curve, linear x-axis
    fig3, ax3 = plt.subplots(figsize=(12, 7))
    ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
    ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
    ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")
    if trigger_epoch is not None:
        trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
        ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                    label=f"Trigger (epoch {trigger_epoch:.0f})")
    ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
    ax3.set_ylabel("Fast MA of Slow MA − Slow MA", fontsize=12)
ax3.set_title(f"4-head model, Run {run_number}: Difference — Linear Scale",
fontsize=13)
    ax3.legend(loc="upper right", fontsize=11)
    ax3.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig3, "ma_of_slow_ma_diff_linear.png")

    # Plot 4: Difference curve overlaid on the grokking curve
    fig4, ax4 = plt.subplots(figsize=(12, 7))
ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5,
label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5,
label="Test Accuracy")
    ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
    ax4.set_ylabel("Accuracy", fontsize=12)
    ax4.set_ylim(-0.05, 1.05)
    ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA
− Slow MA", alpha=0.85)
    ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
    ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)
    if grokked:
ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
    if trigger_epoch is not None:
        ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch
{trigger_epoch:.0f})")
title_suffix = f"Grok epoch {grok_epoch}" if grokked else "did not grok in this run"
ax4.set_title(f"4-head model, Run {run_number}: MA-of-MA Difference vs. Grokking
Curve — {title_suffix}", fontsize=13)
    lines4, labels4 = ax4.get_legend_handles_labels()
    lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
    ax4.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig4, "ma_of_ma_diff_vs_grokking_linear.png")
```

```
    # Plot 5: Grokking curve
    fig5, ax5 = plt.subplots(figsize=(12, 7))
ax5.plot(epochs_axis, train_acc, label="Train Accuracy", color="steelblue",
linewidth=2)
ax5.plot(epochs_axis, test_acc, label="Test Accuracy", color="seagreen",
linewidth=2)
    ax5.set_xscale("log")
    ax5.set_xlabel("Epoch (log scale)")
    ax5.set_ylabel("Accuracy")
ax5.set_title(f"4-head model, Run {run_number}: Grokking Curve — Train vs. Test
Accuracy")
    ax5.legend(loc="center right")
    ax5.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig5, "grokking_curve.png")

    # Plot 6: Loss curve
    fig6, ax6 = plt.subplots(figsize=(12, 7))
    ax6.plot(epochs_axis, loss_history, color="darkorange", linewidth=2)
    ax6.set_xscale("log")
    ax6.set_xlabel("Epoch (log scale)")
    ax6.set_ylabel("Loss")
    ax6.set_title(f"4-head model, Run {run_number}: Training Loss")
    ax6.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig6, "loss_curve.png")

    # Plot 7: L2 Norm curve, with MA Crossover marked
    fig7, ax7 = plt.subplots(figsize=(12, 7))
ax7.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
    if detection_epoch is not None:
        ax7.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                    label=f"MA Crossover (epoch {detection_epoch:.0f})")
    ax7.set_xscale("log")
    ax7.set_xlabel("Epoch (log scale)")
    ax7.set_ylabel("L2 Norm")
    ax7.set_title(f"4-head model, Run {run_number}: L2 Norm of Model Weights")
    ax7.legend(loc="upper right")
    ax7.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig7, "l2_norm_curve.png")

    # Plot 8: Dropout Gap curve
    fig8, ax8 = plt.subplots(figsize=(12, 7))
    ax8.plot(dropout_gap_epochs, dropout_gap_history, color="crimson", linewidth=2)
    ax8.set_xscale("log")
    ax8.set_xlabel("Epoch (log scale)")
    ax8.set_ylabel("Dropout Gap")
    ax8.set_title(f"4-head model, Run {run_number}: Dropout Gap")
    ax8.grid(True, alpha=0.3)
    plt.tight_layout()
    save(fig8, "dropout_gap_curve.png")

    return {
        "run_number": run_number, "num_epochs": num_epochs, "grokked": grokked,
        "grok_epoch": grok_epoch, "final_test_acc": float(test_acc[-1]),
"final_l2_norm": float(l2_norm_history[-1]), "final_dropout_gap":
```

```
float(dropout_gap_history[-1]),
    }


def plot_comparison(all_runs_data, out_dir):
    """Builds 4 plots overlaying every discovered run together, saved
    directly in runs/four_head/ (not inside any single run's folder),
    so all previous runs can be read on one chart."""

    def color_for(i):
        return RUN_COLORS[i % len(RUN_COLORS)]

    # Comparison Plot A: Test Accuracy (grokking curve) across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        color = color_for(i)
        ax.plot(epochs_axis, data["test_acc"], color=color, linewidth=2.2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
        if (data["test_acc"] > 0.9).any():
            grok_epoch = int(np.argmax(data["test_acc"] > 0.9))
ax.axvline(x=grok_epoch, color=color, linestyle=":", linewidth=1.5, alpha=0.6)
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Test Accuracy", fontsize=12)
    ax.set_ylim(-0.05, 1.05)
    ax.set_title("4-head model: Test Accuracy Across All Runs", fontsize=13)
    ax.legend(loc="center right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_grokking_curve.png"), dpi=150,
bbox_inches="tight")
    plt.close(fig)
print("[OK] Comparison plot saved to runs/four_head/comparison_grokking_curve.png")

    # Comparison Plot B: Loss across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
        ax.plot(epochs_axis, data["loss"], color=color_for(i), linewidth=2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Loss", fontsize=12)
    ax.set_title("4-head model: Training Loss Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_loss_curve.png"), dpi=150,
bbox_inches="tight")
    plt.close(fig)
    print("[OK] Comparison plot saved to runs/four_head/comparison_loss_curve.png")

    # Comparison Plot C: L2 Norm across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
        epochs_axis = range(1, data["num_epochs"] + 1)
```

```
        ax.plot(epochs_axis, data["l2_norm"], color=color_for(i), linewidth=2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("L2 Norm", fontsize=12)
ax.set_title("4-head model: L2 Norm of Model Weights Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_l2_norm_curve.png"), dpi=150,
bbox_inches="tight")
    plt.close(fig)
print("[OK] Comparison plot saved to runs/four_head/comparison_l2_norm_curve.png")

    # Comparison Plot D: Dropout Gap across all runs
    fig, ax = plt.subplots(figsize=(12, 7))
    for i, (run_number, data) in enumerate(all_runs_data):
ax.plot(data["dropout_gap_epochs"], data["dropout_gap"], color=color_for(i),
linewidth=2,
                label=f"Run {run_number} ({data['num_epochs']} epochs)")
    ax.set_xscale("log")
    ax.set_xlabel("Epoch (log scale)", fontsize=12)
    ax.set_ylabel("Dropout Gap", fontsize=12)
    ax.set_title("4-head model: Dropout Gap Across All Runs", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
plt.savefig(os.path.join(out_dir, "comparison_dropout_gap_curve.png"), dpi=150,
bbox_inches="tight")
    plt.close(fig)
print("[OK] Comparison plot saved to
runs/four_head/comparison_dropout_gap_curve.png")


migrate_legacy_flat_run(four_head_dir)
run_dirs = discover_run_dirs(four_head_dir)

if not run_dirs:
print(f"No runs found under runs/four_head/. Run train_four_head.py at least once
first.")
else:
print(f"Found {len(run_dirs)} run(s): {', '.join(f'Run {n}' for n, _ in
run_dirs)}\n")

    all_runs_data = []
    summaries = []
    for run_number, run_dir in run_dirs:
        data = load_run_data(run_dir)
        if data is None:
            print(f"  Skipping Run {run_number} (incomplete or empty)")
            continue
        all_runs_data.append((run_number, data))
        summaries.append(plot_single_run(run_number, run_dir, data))

    plot_comparison(all_runs_data, four_head_dir)

    print("\n" + "=" * 60)
```

```
    print("SUMMARY ACROSS ALL RUNS")
    print("=" * 60)
    for s in summaries:
grok_text = f"grokked at epoch {s['grok_epoch']}" if s["grokked"] else "did NOT
grok"
        print(f"  Run {s['run_number']} ({s['num_epochs']} epochs): {grok_text}, "
              f"final test acc = {s['final_test_acc']:.4f}, "
              f"final L2 norm = {s['final_l2_norm']:.4f}, "
              f"final dropout gap = {s['final_dropout_gap']:.4f}")

    print(f"\n[OK] Per-run plots saved inside each runs/four_head/run_<N>/ folder.")
    print(f"[OK] 4 comparison plots saved directly in runs/four_head/:")
    print("  - comparison_grokking_curve.png")
    print("  - comparison_loss_curve.png")
    print("  - comparison_l2_norm_curve.png")
    print("  - comparison_dropout_gap_curve.png")
```

File: src\plot_shadow_layernorm.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

# Look for files in project root (parent of src/), same pattern as plot_results.py
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Known plateau value from the original (no-LayerNorm) model, recorded in
# context.md (Aug 7, 2026 plateau-investigation session): 6557/6587 correct.
ORIGINAL_PLATEAU_ACC = 0.995446

shadow_train_acc = np.load("shadow_ln_train_acc_history.npy")
shadow_test_acc = np.load("shadow_ln_test_acc_history.npy")
num_epochs = len(shadow_train_acc)
epochs_axis = range(1, num_epochs + 1)

fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epochs_axis, shadow_train_acc, color="steelblue", linewidth=2.5,
label="Shadow (LayerNorm) Train Accuracy")
ax.plot(epochs_axis, shadow_test_acc, color="seagreen", linewidth=2.5, label="Shadow
(LayerNorm) Test Accuracy")

# Reference line: the original no-LayerNorm model's known plateau value.
ax.axhline(y=ORIGINAL_PLATEAU_ACC, color="darkred", linestyle="--", linewidth=2,
           label=f"Original plateau (no LayerNorm): {ORIGINAL_PLATEAU_ACC:.4%}")

# If the original run's test_acc_history.npy is present in the project root,
# overlay it too, so both curves can be compared directly on one graph.
# This is optional — the shadow curve above still plots fine without it.
try:
    original_test_acc = np.load("test_acc_history.npy")
    original_epochs_axis = range(1, len(original_test_acc) + 1)
ax.plot(original_epochs_axis, original_test_acc, color="darkorange", linewidth=2,
linestyle=":", label="Original (no LayerNorm) Test Accuracy", alpha=0.85)
```

```
except FileNotFoundError:
    print("Note: test_acc_history.npy (original run) not found — skipping overlay, "
          "shadow curve plotted alone.")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("Accuracy", fontsize=12)
ax.set_ylim(-0.05, 1.05)
ax.set_title("Shadow Model (with LayerNorm) — Grokking Curve vs. Original Plateau",
fontsize=13)
ax.legend(loc="center right", fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("shadow_ln_grokking_curve.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to shadow_ln_grokking_curve.png")

final_shadow_test_acc = shadow_test_acc[-1]
print(f"\nFinal shadow (LayerNorm) test accuracy: {final_shadow_test_acc:.6f}")
print(f"Original (no LayerNorm) plateau:         {ORIGINAL_PLATEAU_ACC:.6f}")
if final_shadow_test_acc > ORIGINAL_PLATEAU_ACC:
    print("Shadow model beat the original plateau.")
elif final_shadow_test_acc == ORIGINAL_PLATEAU_ACC:
    print("Shadow model matched the original plateau exactly.")
else:
    print("Shadow model did not reach the original plateau.")
```

File: src\predictors\dropout.py

```
import torch


def compute_accuracy(model, data_loader):
    device = next(model.parameters()).device
    total_correct = 0
    total_samples = 0
    for x, y in data_loader:
        x, y = x.to(device), y.to(device)
        with torch.no_grad():
            logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]
        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)
    return total_correct / total_samples if total_samples > 0 else 0.0


def compute_dropout_gap(model, data_loader, dropout_rate):
    model.train()
    model.dropout1.p = dropout_rate
    model.dropout2.p = dropout_rate
    train_accuracy = compute_accuracy(model, data_loader)

    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0
    eval_accuracy = compute_accuracy(model, data_loader)
```

```
    return train_accuracy - eval_accuracy, train_accuracy, eval_accuracy

def compute_dropout_gap_multi_rate(model, data_loader, dropout_rates):
    # Step 1: START (inputs: model, data_loader, dropout_rates)

    # Step 2: Setup Evaluation Mode
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 3: Compute Clean Accuracy (execute ONLY ONCE)
    clean_accuracy = compute_accuracy(model, data_loader)

    # Step 4: Initialize Results Storage
    results = {}

    # Step 5: FOR each rate in dropout_rates
    for dropout_rate in dropout_rates:
        # --- Switch to train mode with dropout active ---
        model.train()
        model.dropout1.p = dropout_rate
        model.dropout2.p = dropout_rate

        # Compute train accuracy (with dropout)
        train_accuracy = compute_accuracy(model, data_loader)

        # Compute dropout gap (train_accuracy - clean_accuracy)
        dropout_gap = train_accuracy - clean_accuracy

        # Store results
        results[dropout_rate] = {
            "train_accuracy": train_accuracy,
"eval_accuracy": clean_accuracy,      # == clean_accuracy (dropout=0, eval mode)
            "dropout_gap": dropout_gap,
            "clean_accuracy": clean_accuracy
        }

    # Step 6: Loop Decision — "More rates?" (handled by for-loop)

    # Step 7: Final Cleanup — restore model to clean eval state
    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0

    # Step 8: RETURN results — END
    return results

File: src\predictors\l2_norm.py

import numpy as np
import torch
from scipy.ndimage import uniform_filter1d

def compute_l2_norm(model):
    all_params = torch.cat([p.flatten() for p in model.parameters()])
    l2_norm = torch.norm(all_params).item()
```

```
    return l2_norm

def compute_l2_norm_rate_of_decline(l2_norm_history):
    return np.diff(l2_norm_history) *-1


def detect_l2_norm_drop(rate_of_decline, threshold=0.05, skip_epochs=200):
    for i, rate in enumerate(rate_of_decline):
        if i < skip_epochs:  # Skip early epochs
            continue
        if rate > threshold:
            return i
    return None


def compute_acceleration(rate_of_decline):
    """
    Compute the second derivative of L2 norm (acceleration).
    acceleration[i] = rate[i] - rate[i-1]
    Positive = decay rate is increasing (speeding up)
    Negative = decay rate is decreasing (slowing down)
    Sign change = inflection point (change in decay character)
    """
    acceleration = np.diff(rate_of_decline)
    return acceleration


def detect_inflection(acceleration, skip_epochs=200):
    """
    Detect inflection point where acceleration changes sign.
    This is where the decay pattern changes character — exactly what happens
    before/during the grok transition.
    Returns the epoch index where the sign flip occurs, or None.
    """
    if len(acceleration) <= skip_epochs:
        return None

    # Check from skip_epochs onwards for sign changes
    for i in range(skip_epochs, len(acceleration)):
        # acceleration[i-1] * acceleration[i] < 0 means opposite signs (sign flip)
        if i > 0 and acceleration[i-1] * acceleration[i] < 0:
            return i

    return None


def apply_moving_average(data, window_size=50):
    """
    Apply moving average smoothing to reduce noise.
    window_size: number of epochs to average over (default 50).
    Returns smoothed array (same length as input, padded at edges).
    """
    return uniform_filter1d(np.array(data), size=window_size, mode='nearest')


def resample_to_log_uniform_grid(data, num_points=None):
    """
```

```
    Resample data onto a grid uniformly spaced in log10(epoch) — i.e. the
    SAME spacing the data has when viewed on a log-x plot.
    Without this, a fixed window_size (in raw epoch index) covers a huge
    visual chunk of the plot near epoch 1 and a tiny sliver near epoch N,
    which is why smoothing looked over-smoothed early and scribbly late.
    Returns (epoch_grid, resampled_data) where epoch_grid is uniform in
    log10-space (non-uniform in linear epoch terms).
    """
    data = np.array(data)
    num_epochs = len(data)
    epochs = np.arange(1, num_epochs + 1)
    log_epochs = np.log10(epochs)

    if num_points is None:
        num_points = num_epochs

    log_grid = np.linspace(log_epochs[0], log_epochs[-1], num_points)
    resampled = np.interp(log_grid, log_epochs, data)
    epoch_grid = 10 ** log_grid

    return epoch_grid, resampled


def compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,
slow_window=200, num_points=None):
    """
    Compute two moving averages on a LOG-EPOCH-UNIFORM grid, so the window
    covers equal VISUAL width everywhere on a log-x plot (fixes both the
    over-smoothing near epoch 1 and the scribbling near epoch N).

    Steps:
      1. Log-transform L2 norm values (handles the exponential decay in y).
      2. Resample onto a grid uniform in log10(epoch) (handles the log-x axis).
      3. Apply the moving average on that resampled grid.
      4. Convert back to linear L2 norm space.

    fast_window / slow_window are now measured in grid points, which (since
    the grid is log-uniform) corresponds to a constant *proportional* width
    in epochs everywhere along the curve.

    Returns (epoch_grid, fast_ma, slow_ma) — always plot fast_ma/slow_ma
    against epoch_grid, not against range(1, num_epochs+1).
    """
    l2_norm_history = np.array(l2_norm_history)
    log_l2_norm = np.log(l2_norm_history)

epoch_grid, log_l2_norm_resampled = resample_to_log_uniform_grid(log_l2_norm,
num_points=num_points)

fast_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=fast_window)
slow_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=slow_window)

    fast_ma = np.exp(fast_ma_log)
    slow_ma = np.exp(slow_ma_log)

    return epoch_grid, fast_ma, slow_ma
```

```
def detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100):
    """
    Detect where fast MA crosses above slow MA, searching along the
    log-uniform grid. skip_epochs is a real epoch value (not a grid index) —
    crossovers before this epoch are ignored (initialization noise).
    Returns the real epoch (float) of the crossover, or None.
    """
    if len(fast_ma) != len(slow_ma):
        return None

    for i in range(len(fast_ma) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        # Previous: fast <= slow, Current: fast > slow (crossover from below)
        if fast_ma[i] <= slow_ma[i] and fast_ma[i + 1] > slow_ma[i + 1]:
            return epoch_grid[i + 1]

    return None


def compute_ma_of_slow_ma(slow_ma, fast_window=20):
    """
    Apply a second, faster-window moving average on top of the already-smoothed
    slow_ma. Since slow_ma has already filtered out raw L2 norm noise, this
    reveals slow_ma's OWN turning points cleanly — the difference between the
    two tracks how sharply slow_ma is currently bending.
    Returns (fast_ma_of_slow_ma, diff) where diff = fast_ma_of_slow_ma - slow_ma.
    """
    fast_ma_of_slow_ma = apply_moving_average(slow_ma, window_size=fast_window)
    diff = fast_ma_of_slow_ma - slow_ma
    return fast_ma_of_slow_ma, diff


def compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=90):
    """
    Estimate the normal noise level of the MA-of-MA difference from the quiet
    early-training region (before any real dynamics begin), so later spikes
    can be judged against what "flat" actually looks like for this run.
    """
    quiet_mask = epoch_grid < quiet_epoch_cutoff
    return np.std(diff[quiet_mask])


def detect_ma_of_ma_trigger(epoch_grid, diff, noise_floor, threshold_multiplier=10,
skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff climbs past
    threshold_multiplier x noise_floor — the rising edge of a real signal.
    Causal: only needs the noise floor (known early) and points seen so far,
    unlike picking the curve's peak, which requires seeing the whole future.
    Returns the real epoch (float) of the trigger, or None.
    """
    threshold = threshold_multiplier * noise_floor
    for i in range(len(diff)):
        if epoch_grid[i] < skip_epochs:
            continue
```

```
        if diff[i] > threshold:
            return epoch_grid[i]
    return None


def detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff crosses from
    positive to negative (the first "green -> red" zero crossing).
    Confirmed on two separate training runs to land in a consistent,
    unambiguous spot even though the crossing's magnitude is weak.
    Returns the real epoch (float) of the crossing, or None.
    """
    for i in range(len(diff) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] >= 0 and diff[i + 1] < 0:
            return epoch_grid[i + 1]
    return None


File: src\train.py
```

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_dataloaders
from models.transformer import Transformer
from predictors.dropout import compute_dropout_gap_multi_rate

# Save results to project root
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)



device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_dataloaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
```

```
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
dropout_gap_epochs = []
dropout_gap_history = []
dropout_train_acc_history = []
dropout_eval_acc_history = []
dropout_rates = [0.1, 0.3, 0.5, 0.7, 0.9]
dropout_gap_history_by_rate = {rate: [] for rate in dropout_rates}

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))

    # Dropout Gap Predictor: now computed at every epoch (previously throttled
    # to every 100th epoch, because it needs two full passes over the test set
    # — dropout ON, then dropout OFF — which is costly). Jonathan has since
    # asked for full per-epoch resolution here as well, so this block now
    # runs unconditionally. This will make each epoch noticeably slower than
    # before, since two extra passes over the test set now happen every time.
# Now tracking multi-rate dropout gaps across five rates: [0.1, 0.3, 0.5, 0.7, 0.9]
    results = compute_dropout_gap_multi_rate(
        model, data_loader[1], dropout_rates
    )

    for rate in dropout_rates:
```

```
        dropout_gap_history_by_rate[rate].append(
            results[rate]["dropout_gap"]
        )

    # Maintain backward compatibility with single-rate record (rate=0.9)
    dropout_gap = results[0.9]["dropout_gap"]
    dropout_train_acc = results[0.9]["train_accuracy"]
    dropout_eval_acc = results[0.9]["eval_accuracy"]
    dropout_gap_epochs.append(epoch + 1)
    dropout_gap_history.append(dropout_gap)
    dropout_train_acc_history.append(dropout_train_acc)
    dropout_eval_acc_history.append(dropout_eval_acc)

    # compute_dropout_gap() leaves the model in eval() mode with p reset to
    # 0.0. Training must resume in train() mode, so we restore it here before
    # the next epoch's training pass begins.
    model.train()

print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
{train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
{l2_norm_history[-1]:.4f}, Dropout Gap: {dropout_gap:.4f}")

np.save("train_acc_history.npy", train_acc_history)
np.save("test_acc_history.npy", test_acc_history)
np.save("loss_history.npy", loss_history)
np.save("l2_norm_history.npy", l2_norm_history)
np.save("dropout_gap_epochs.npy", dropout_gap_epochs)
np.save("dropout_gap_history.npy", dropout_gap_history)
np.save("dropout_train_acc_history.npy", dropout_train_acc_history)
np.save("dropout_eval_acc_history.npy", dropout_eval_acc_history)
print("Training data saved:")
print("  - train_acc_history.npy")
print("  - test_acc_history.npy")
print("  - loss_history.npy")
print("  - l2_norm_history.npy")
print("  - dropout_gap_epochs.npy")
print("  - dropout_gap_history.npy")
print("  - dropout_train_acc_history.npy")
print("  - dropout_eval_acc_history.npy")



# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: Moving Average Crossover")
print("="*60)

# Compute fast and slow moving averages on a log-epoch-uniform grid
# (matches the log-x plot's visual spacing, so the window covers equal
# visual width everywhere instead of scribbling at high epochs)
# Fast MA (window=50 grid points): responsive to recent changes
# Slow MA (window=200 grid points): captures overall trend
epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history,
fast_window=50, slow_window=200)

# Detect crossover point: where fast MA crosses above slow MA
detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)
```

```
# Find when test accuracy actually jumps (crosses 90%)
grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

# Save data for plotting
np.save("epoch_grid.npy", epoch_grid)
np.save("fast_ma.npy", fast_ma)
np.save("slow_ma.npy", slow_ma)

# Report results
print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

# Debug stats
print(f"\nL2 Norm Stats:")
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")


# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: MA of Slow MA — Zero-Crossing Trigger")
print("="*60)

# Apply a second, faster-window MA on top of slow_ma itself. slow_ma has
# already filtered out raw L2 norm noise, so this reveals slow_ma's own
# turning points cleanly instead of chasing noise in the raw signal.
fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

# Noise floor: how big this difference is during the quiet early-training
# region, before any real dynamics begin — printed for context only, no
# longer used to decide the trigger (see detect_ma_of_ma_zero_crossing).
noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

# Trigger: first epoch (after skip_epochs) where the difference crosses from
# positive to negative. Confirmed on two runs to land in a consistent,
# unambiguous spot even though the crossing itself is weak in magnitude.
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff,
skip_epochs=100)

np.save("fast_ma_of_slow_ma.npy", fast_ma_of_slow_ma)
np.save("ma_of_ma_diff.npy", ma_of_ma_diff)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f"  Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {trigger_lead_time:.1f} epochs")
```

```
else:
    print("  No trigger detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")



# Dropout Predictor: gap tracked at every epoch during training (see loop
# above). We simply report the last tracked value here — no need to run
# compute_dropout_gap() a second time on the already-trained model.
print("\n" + "="*60)
print("DROPOUT PREDICTOR: Gap tracked every epoch")
print("="*60)
print(f"\nFinal Dropout Gap Check (rate=0.9, epoch {dropout_gap_epochs[-1]}):")
print(f"  Accuracy with dropout (train mode, p=0.9):
{dropout_train_acc_history[-1]:.4f}")
print(f"  Accuracy without dropout (eval mode, p=0.0):
{dropout_eval_acc_history[-1]:.4f}")
print(f"  Gap: {dropout_gap_history[-1]:.4f}")


# ======================================================================
# PDF REPORT GENERATION
# ======================================================================
# matplotlib is imported only here, at the very end, after training is
# fully finished. 'Agg' backend must be selected before importing pyplot,
# otherwise matplotlib tries to load a DLL that Windows blocks on this
# machine — same workaround already confirmed working in
# src/plot_results.py.
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

print("\n" + "="*60)
print("PDF REPORT GENERATION")
print("="*60)

epochs_axis = range(1, num_epochs + 1)

with PdfPages("training_report.pdf") as pdf:

    # Page 1: Grokking curve — Train vs. Test accuracy
    fig1, ax1 = plt.subplots(figsize=(12, 7))
ax1.plot(epochs_axis, train_acc_history, label="Train Accuracy", color="steelblue",
linewidth=2)
ax1.plot(epochs_axis, test_acc_history, label="Test Accuracy", color="seagreen",
linewidth=2)
    ax1.set_xscale("log")
    ax1.set_xlabel("Epoch (log scale)")
    ax1.set_ylabel("Accuracy")
    ax1.set_title("Grokking Curve: Train vs. Test Accuracy")
    ax1.legend(loc="center right")
    ax1.grid(True, alpha=0.3)
    pdf.savefig(fig1)
    plt.close(fig1)

    # Page 2: Loss curve
```

```
    fig2, ax2 = plt.subplots(figsize=(12, 7))
    ax2.plot(epochs_axis, loss_history, color="darkorange", linewidth=2)
    ax2.set_xscale("log")
    ax2.set_xlabel("Epoch (log scale)")
    ax2.set_ylabel("Loss")
    ax2.set_title("Training Loss")
    ax2.grid(True, alpha=0.3)
    pdf.savefig(fig2)
    plt.close(fig2)

    # Page 3: L2 norm curve, with the MA crossover detection epoch marked
    # (detection_epoch was computed earlier in this same script, in the
    # L2 Norm Predictor section).
    fig3, ax3 = plt.subplots(figsize=(12, 7))
ax3.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
    if detection_epoch is not None:
        ax3.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                    label=f"MA Crossover (epoch {detection_epoch:.0f})")
    ax3.set_xscale("log")
    ax3.set_xlabel("Epoch (log scale)")
    ax3.set_ylabel("L2 Norm")
    ax3.set_title("L2 Norm of Model Weights")
    ax3.legend(loc="upper right")
    ax3.grid(True, alpha=0.3)
    pdf.savefig(fig3)
    plt.close(fig3)

    # Page 4: Dropout Gap — now recorded at every epoch, same resolution as
    # the other three plots, so it uses the same log-x style for consistency.
    fig4, ax4 = plt.subplots(figsize=(12, 7))
    ax4.plot(dropout_gap_epochs, dropout_gap_history, color="crimson", linewidth=2)
    ax4.set_xscale("log")
    ax4.set_xlabel("Epoch (log scale)")
    ax4.set_ylabel("Dropout Gap")
    ax4.set_title("Dropout Gap")
    ax4.grid(True, alpha=0.3)
    pdf.savefig(fig4)
    plt.close(fig4)

print(f"[OK] PDF report saved to training_report.pdf (4 pages: 4 graphs)")
```

File: src\train_four_head.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
import re
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
```

```
from data.modular_arithmetic import get_dataloaders
from models.transformer_four_head import TransformerFourHead
from predictors.dropout import compute_dropout_gap

# ======================================================================
# 4-HEAD VARIANT of train.py.
#
# This script is a separate copy of the main training loop — it does NOT
# modify train.py or any of its saved outputs. The only real difference
# from train.py is the model: TransformerFourHead(num_heads=4) instead of
# the original single-head Transformer. Same task (p=97), same optimiser
# (AdamW, lr=1e-3, weight_decay=1.0), same full-batch training, same 30/70
# split, same L2 Norm and Dropout Gap predictor code (reused as-is from
# src/predictors/ — nothing predictor-specific needed to change, since
# both predictors work on any model that exposes .parameters() and
# .dropout1/.dropout2, which this model does too).
#
# Outputs are saved under runs/four_head/run_<N>/ (created if it does not
# exist yet), NOT in the project root, so they never overwrite train.py's
# own train_acc_history.npy / l2_norm_history.npy / dropout_gap_history.npy
# / etc., which belong to the original single-head run.
#
# RUN NUMBERING: every time this script is run, it saves into a NEW
# numbered subfolder (run_1, run_2, run_3, ...) instead of overwriting the
# previous run's results. This is needed because grokking is stochastic —
# the grok epoch moves with the random seed — so a real comparison needs
# several independent runs kept side by side, not one run's numbers
# overwritten by the next. plot_results_four_head.py reads every run_<N>
# folder it finds and plots them together for comparison.
# ======================================================================

LEGACY_FILENAMES = [
    "train_acc_history.npy", "test_acc_history.npy", "loss_history.npy",
    "l2_norm_history.npy", "dropout_gap_epochs.npy", "dropout_gap_history.npy",
    "dropout_train_acc_history.npy", "dropout_eval_acc_history.npy",
    "epoch_grid.npy", "fast_ma.npy", "slow_ma.npy", "fast_ma_of_slow_ma.npy",
    "ma_of_ma_diff.npy", "training_report.pdf",
    "ma_of_slow_ma_crossover.png", "ma_of_slow_ma_diff.png",
    "ma_of_slow_ma_diff_linear.png", "ma_of_ma_diff_vs_grokking_linear.png",
"grokking_curve.png", "loss_curve.png", "l2_norm_curve.png",
"dropout_gap_curve.png",
]


def migrate_legacy_flat_run(four_head_dir):
    """
    Earlier versions of this script saved directly into runs/four_head/
    instead of runs/four_head/run_<N>/. If such files are found here, and
    run_1/ does not exist yet, move them into run_1/ so they are counted
    as the first run instead of silently sitting outside the new
    numbering scheme (and instead of being silently overwritten by this
    run, which is what would have happened before this change).
    """
    run_1_dir = os.path.join(four_head_dir, "run_1")
    if os.path.isdir(run_1_dir):
        return
    legacy_present = any(
```

```
os.path.isfile(os.path.join(four_head_dir, name)) for name in LEGACY_FILENAMES
    )
    if not legacy_present:
        return
    os.makedirs(run_1_dir, exist_ok=True)
    for name in LEGACY_FILENAMES:
        src = os.path.join(four_head_dir, name)
        if os.path.isfile(src):
            os.rename(src, os.path.join(run_1_dir, name))
print("[MIGRATION] Found results saved directly in runs/four_head/ by an earlier "
"version of this script — moved them into runs/four_head/run_1/ so they are "
          "counted as Run 1 instead of being overwritten.")


def get_next_run_number(four_head_dir):
    if not os.path.isdir(four_head_dir):
        return 1
    existing_numbers = []
    for name in os.listdir(four_head_dir):
        match = re.fullmatch(r"run_(\d+)", name)
        if match and os.path.isdir(os.path.join(four_head_dir, name)):
            existing_numbers.append(int(match.group(1)))
    return max(existing_numbers, default=0) + 1


project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

four_head_dir = os.path.join(project_root, "runs", "four_head")
migrate_legacy_flat_run(four_head_dir)
run_number = get_next_run_number(four_head_dir)

output_dir = os.path.join(four_head_dir, f"run_{run_number}")
os.makedirs(output_dir, exist_ok=True)
os.chdir(output_dir)

print(f"This is Run {run_number}. All outputs will be saved to
runs/four_head/run_{run_number}/")

# NEW random seed on every run, instead of a fixed one. torch.seed() picks
# a fresh, non-deterministic seed itself and also returns the exact number
# it used, so we can both use it and record it in the same step. This is
# needed because grokking is stochastic (grok epoch moves with the seed —
# see the L2 Norm predictor's 3-run history) and the thesis needs several
# genuinely independent 4-head runs, not the same run repeated.
seed = torch.seed()
print(f"Seed: {seed}")
np.save("seed.npy", seed)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_dataloaders(97, batch_size=int(0.3 * 97 * 97))
model = TransformerFourHead(vocab_size=98, d_model=128, num_heads=4).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Model: TransformerFourHead (num_heads=4, matches Nanda et al.)")
```

```
print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 40000
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
dropout_gap_epochs = []
dropout_gap_history = []
dropout_train_acc_history = []
dropout_eval_acc_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))

    # Dropout Gap Predictor: same per-epoch measurement as train.py,
    # reusing compute_dropout_gap unchanged — it only needs model.dropout1
    # / model.dropout2, which this 4-head model also has.
    dropout_gap, dropout_train_acc, dropout_eval_acc = compute_dropout_gap(
        model, data_loader[1], dropout_rate=0.9
    )
    dropout_gap_epochs.append(epoch + 1)
    dropout_gap_history.append(dropout_gap)
    dropout_train_acc_history.append(dropout_train_acc)
    dropout_eval_acc_history.append(dropout_eval_acc)
```

```
    # compute_dropout_gap() leaves the model in eval() mode with p reset
    # to 0.0. Training must resume in train() mode, so we restore it here
    # before the next epoch's training pass begins.
    model.train()

print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
{train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
{l2_norm_history[-1]:.4f}, Dropout Gap: {dropout_gap:.4f}")

np.save("train_acc_history.npy", train_acc_history)
np.save("test_acc_history.npy", test_acc_history)
np.save("loss_history.npy", loss_history)
np.save("l2_norm_history.npy", l2_norm_history)
np.save("dropout_gap_epochs.npy", dropout_gap_epochs)
np.save("dropout_gap_history.npy", dropout_gap_history)
np.save("dropout_train_acc_history.npy", dropout_train_acc_history)
np.save("dropout_eval_acc_history.npy", dropout_eval_acc_history)
print(f"Training data saved (runs/four_head/run_{run_number}/):")
print("  - train_acc_history.npy")
print("  - test_acc_history.npy")
print("  - loss_history.npy")
print("  - l2_norm_history.npy")
print("  - dropout_gap_epochs.npy")
print("  - dropout_gap_history.npy")
print("  - dropout_train_acc_history.npy")
print("  - dropout_eval_acc_history.npy")



# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model): Moving Average Crossover")
print("="*60)

epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history,
fast_window=50, slow_window=200)

detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

np.save("epoch_grid.npy", epoch_grid)
np.save("fast_ma.npy", fast_ma)
np.save("slow_ma.npy", slow_ma)

print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

print(f"\nL2 Norm Stats:")
```

```
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")


# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR (4-head model): MA of Slow MA — Zero-Crossing Trigger")
print("="*60)

fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff,
skip_epochs=100)

np.save("fast_ma_of_slow_ma.npy", fast_ma_of_slow_ma)
np.save("ma_of_ma_diff.npy", ma_of_ma_diff)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f"  Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {trigger_lead_time:.1f} epochs")
else:
    print("  No trigger detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")



# Dropout Predictor: gap tracked at every epoch during training (see loop
# above). We simply report the last tracked value here.
print("\n" + "="*60)
print("DROPOUT PREDICTOR (4-head model): Gap tracked every epoch")
print("="*60)
print(f"\nFinal Dropout Gap Check (rate=0.9, epoch {dropout_gap_epochs[-1]}):")
print(f"  Accuracy with dropout (train mode, p=0.9):
{dropout_train_acc_history[-1]:.4f}")
print(f"  Accuracy without dropout (eval mode, p=0.0):
{dropout_eval_acc_history[-1]:.4f}")
print(f"  Gap: {dropout_gap_history[-1]:.4f}")


# ======================================================================
# PDF REPORT GENERATION
# ======================================================================
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

print("\n" + "="*60)
print("PDF REPORT GENERATION (4-head model)")
print("="*60)
```

```
epochs_axis = range(1, num_epochs + 1)

with PdfPages("training_report.pdf") as pdf:

    fig1, ax1 = plt.subplots(figsize=(12, 7))
ax1.plot(epochs_axis, train_acc_history, label="Train Accuracy", color="steelblue",
linewidth=2)
ax1.plot(epochs_axis, test_acc_history, label="Test Accuracy", color="seagreen",
linewidth=2)
    ax1.set_xscale("log")
    ax1.set_xlabel("Epoch (log scale)")
    ax1.set_ylabel("Accuracy")
    ax1.set_title("Grokking Curve (4-head model): Train vs. Test Accuracy")
    ax1.legend(loc="center right")
    ax1.grid(True, alpha=0.3)
    pdf.savefig(fig1)
    plt.close(fig1)

    fig2, ax2 = plt.subplots(figsize=(12, 7))
    ax2.plot(epochs_axis, loss_history, color="darkorange", linewidth=2)
    ax2.set_xscale("log")
    ax2.set_xlabel("Epoch (log scale)")
    ax2.set_ylabel("Loss")
    ax2.set_title("Training Loss (4-head model)")
    ax2.grid(True, alpha=0.3)
    pdf.savefig(fig2)
    plt.close(fig2)

    fig3, ax3 = plt.subplots(figsize=(12, 7))
ax3.plot(epochs_axis, l2_norm_history, color="purple", linewidth=2, label="L2 Norm")
    if detection_epoch is not None:
        ax3.axvline(x=detection_epoch, color="red", linestyle="--", linewidth=2,
                    label=f"MA Crossover (epoch {detection_epoch:.0f})")
    ax3.set_xscale("log")
    ax3.set_xlabel("Epoch (log scale)")
    ax3.set_ylabel("L2 Norm")
    ax3.set_title("L2 Norm of Model Weights (4-head model)")
    ax3.legend(loc="upper right")
    ax3.grid(True, alpha=0.3)
    pdf.savefig(fig3)
    plt.close(fig3)

    fig4, ax4 = plt.subplots(figsize=(12, 7))
    ax4.plot(dropout_gap_epochs, dropout_gap_history, color="crimson", linewidth=2)
    ax4.set_xscale("log")
    ax4.set_xlabel("Epoch (log scale)")
    ax4.set_ylabel("Dropout Gap")
    ax4.set_title("Dropout Gap (4-head model)")
    ax4.grid(True, alpha=0.3)
    pdf.savefig(fig4)
    plt.close(fig4)

print(f"[OK] PDF report saved to runs/four_head/run_{run_number}/training_report.pdf
(4 pages: 4 graphs)")

File: src\train_shadow_layernorm.py
```

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os

from data.modular_arithmetic import get_dataloaders
from models.model_shadow_with_layernorm import Transformer

# Save results to project root (same pattern as train.py)
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Fixed seed, so this run can later be repeated with the same starting
# weights and the same train/test split. NOTE: train.py itself does not
# currently set any seed. If you want a true apples-to-apples comparison
# against the original model, add this same line to train.py yourself
# before that run too (temporarily) - this script does not modify train.py.
torch.manual_seed(1337)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)

data_loader = get_dataloaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
train_acc_history = []
test_acc_history = []
loss_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
```

```
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())

    if (epoch + 1) % 100 == 0 or epoch == 0:
        print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, "
              f"Train Accuracy: {train_acc_history[-1]:.4f}, "
              f"Test Accuracy: {test_acc_history[-1]:.4f}")

# Saved under a "shadow_ln_" prefix on purpose, so these files never
# overwrite train.py's own train_acc_history.npy / test_acc_history.npy /
# loss_history.npy, which the L2 Norm and Dropout predictors depend on.
np.save("shadow_ln_train_acc_history.npy", train_acc_history)
np.save("shadow_ln_test_acc_history.npy", test_acc_history)
np.save("shadow_ln_loss_history.npy", loss_history)

final_test_acc = test_acc_history[-1]
test_acc_array = np.array(test_acc_history)
grok_epoch = int(np.argmax(test_acc_array > 0.9)) if (test_acc_array > 0.9).any()
else None

print("\n" + "=" * 60)
print("SHADOW MODEL (LayerNorm) RESULT")
print("=" * 60)
print(f"Final Test Accuracy: {final_test_acc:.4f}")
print(f"Grok epoch (test acc > 90%): {grok_epoch}")
print("Compare this Final Test Accuracy against the original model's known")
print("plateau value: 0.995446 (99.5446%) - see context.md, Aug 7 session.")
```